

COMP634 – Assessment 2 Report

Architectural Image Classification Using Custom and Pretrained CNNs

1. Experimental Protocol

The dataset contains a total of 11,534 architectural images, which were resized to 64×64 for training, and are unevenly divided across 10 architectural classes. We adopted an 80/10/10 train–validation–test split, stratified to preserve the class distribution. This was implemented using a two-stage split (80 percent training, followed by a 50/50 split of the remaining data into validation and test sets). All experiments use the same fixed split to ensure comparability and reproducibility.

Training was conducted using mini-batch SGD via AdamW, mixed-precision, and early stopping based on validation loss with a patience of 4 epochs and a minimum improvement threshold $\delta = 1e-4$.

Three models were trained:

1. **TinyDSCNN** (custom CNN, from scratch)
2. **ResNet18** (existing CNN, trained from scratch)
3. **ResNet18** (existing CNN, pretrained + fine-tuning)

Each model is trained for up to 18 epochs, with each batch learning rate scheduling. All models are evaluated on the out test set, and final metrics were computed from these predictions.

Reproducibility notes

Even with a fixed random seed, deep learning experiments can still show small variations from run to run. This is mainly due to factors like GPU nondeterminism, random data augmentations, and the dynamic learning-rate schedule used by OneCycleLR. Despite these minor fluctuations, the training behaviour and final results remain stable and consistent overall. To keep the evaluation fair and reproducible, we set a fixed seed (42) for the dataset split and used the same train/validation/test partition for all models.

2. Data Pre-processing and Data Augmentation

We used the standard ImageNet normalization statistics (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]) for all models to ensure compatibility with the pretrained ResNet18 pipeline.

2.1 Training Transformations

- RandomHorizontalFlip ($p = 0.5$) — introduces left–right invariance.
- RandomRotation ($\pm 10^\circ$) with $p = 0.25$ — improves robustness to small viewpoint changes.
- ColorJitter (brightness 0.1, contrast 0.1, saturation 0.1) with $p = 0.25$, simulates mild lighting variation.
- ToTensor() — converts pixel values to PyTorch tensors.
- Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]) uses the standard ImageNet statistics for all models.

2.2 Validation/Test Transformations

- Resize to 64×64 , ToTensor(), and the same ImageNet normalization as above

2.3 Handling Class Imbalance

Class counts varied substantially. We computed inverse-frequency class weights and used them in the CrossEntropyLoss. We did not use oversampling to avoid distorting the distribution in each mini-batch.

3. Model Architectures and Training Strategies

Each model was chosen to answer a specific question about the dataset and the classification task.

TinyDSCNN serves as a deliberately efficient baseline designed around the constraints of 64×64 images, allowing us to test whether a lightweight model with depthwise-separable convolutions can extract sufficient spatial structure.

ResNet18 from scratch provides a capacity-controlled baseline that tests how well a modern CNN can learn architectural features purely from this dataset without external priors.

Finally, ResNet18 with pretrained ImageNet weights evaluates the benefit of transfer learning. Because many architectural patterns (arches, columns, vaults, edges) share low- and mid-level features with natural images, pretrained filters are expected to dramatically improve generalisation.

3.1 Custom CNN: TinyDSCNN

TinyDSCNN is a lightweight architecture designed specifically for this assignment. Its design goals were:

- **<1M parameters**
- **Fast training**
- **Efficient on 64×64 images**
- **Resistant to overfitting**

The model uses:

- A convolutional stem
- **6 Depthwise-Separable Convolution Blocks**
- **Squeeze-and-Excitation (SE) modules**
- GELU activations
- Global average pooling + dropout + linear classifier

Depthwise-separable convolutions reduce computation by ≈ 70 percent, allowing the model to achieve strong accuracy with minimal parameters (~ 0.15 M).

Training used:

- AdamW ($\text{lr} = 3\text{e-}3$, weight decay = $1\text{e-}2$)
- OneCycleLR scheduler
- Mixed precision
- Early stopping

3.2 Existing CNN: ResNet18

We evaluated ResNet18 under two strategies.

Strategy 1: ResNet18 (Trained from Scratch)

- Random weight initialisation
- All layers trained
- Same preprocessing pipeline as TinyDSCNN

This strategy tests the baseline capacity of ResNet18 without external knowledge.

Strategy 2: ResNet18 (Pretrained + Fine-Tuning)

- Loaded **ImageNet pretrained weights**
- Replaced final fully-connected layer
- Fine-tuned **all layers**
- Lower learning rate ($1e-4$)

Even though ImageNet models typically require mean/std = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225), we chose to maintain the same normalization across all models for consistency. This may slightly limit peak performance but simplifies the experimental setup.

4. Training Components

- **Loss:** Weighted CrossEntropyLoss (to balance classes)
- **Optimiser:** AdamW
- **Learning rate scheduling:** OneCycleLR (TinyDSCNN), CosineAnnealingLR (ResNet)
- **Gradient scaling:** torch.cuda.amp (mixed precision)
- **Early stopping:** validation loss, patience = 4

The selected hyperparameters were based on empirical evidence about what works well for compact CNNs and ResNet-style architectures. TinyDSCNN uses OneCycleLR because it accelerates convergence for smaller models with rapidly changing feature hierarchies. ResNet uses CosineAnnealingLR, which provides smoother long-term decay suitable for deeper architectures.

The learning rates ($3e-3$ for TinyDSCNN, $3e-4$ / $1e-4$ for ResNet) were chosen through exploratory runs that balanced rapid initial learning with later stability. Early stopping with patience=4 was used to prevent overfitting without prematurely halting training.

5. Results and Comparison

5.1 Performance Summary Table

Model	Strategy	Params (M)	Val Acc	Test Acc	Macro F1	Notes
TinyDSCNN	Scratch	0.150M	0.881	0.893	0.881	Lightweight, efficient custom CNN
ResNet18	Scratch	11.182M	0.826	0.822	0.805	Overfits on small data; weaker baseline
ResNet18	Pretrained + FT	11.182M	0.929	0.932	0.919	Best model; transfer learning gives major gains

5.2 Interpretation

- TinyDSCNN performs impressively well given its size (<1 percent of ResNet's parameters).
- ResNet18 from scratch shows significant overfitting despite high train accuracy (~97 percent).
- Pretrained ResNet18 achieves the **highest performance** due to strong initialisation from ImageNet, allowing rapid convergence (within ~6 epochs).

Thus, **ResNet18 (pretrained + fine-tuning)** is the final selected model.

6. Final Evaluation on Test Set (Best Model)

Accuracy alone can often be misleading under the class imbalance, since it may overrepresent dominant classes. So therefore, macro F1 was included to weight each class equally, making performance on minority classes more visible.

Top-2 accuracy is also meaningful for visually similar architectural categories where multiple structures may coexist within a single image. A model that consistently ranks the correct class in its top two predictions exhibits strong discriminative ability even in ambiguous scenarios.

The final ResNet18 (pretrained) achieved:

- **Top-1 Accuracy: 0.9315**
- **Top-2 Accuracy: 0.9801**
- **Macro F1 Score: 0.919**
- **Weighted F1 Score: 0.932**

These metrics capture different aspects of performance (single-label accuracy, class ranking capability, class balance, and overall multi-class quality).

7. Confusion Matrix and Discussion

7.1 Key Observations

High-performing classes:

- stained_glass (F1 = 0.991)
- gargoyle (0.952)
- dome(inner) (0.971)
- column (0.940)

These classes have strong structural and textural cues which ResNet recognises well.

Most difficult classes:

- apse (F1 = 0.818)
- flying_buttress (0.820)

These categories have lower sample sizes and share features with architecturally similar classes.

Confusions:

- Some overlap occurs between bell_tower and column, likely due to vertical geometric structures.
- apse misclassifications are mostly with altar, as both appear in church interiors.

7.2 Final Confusion Matrix (Normalized)

Include the figure from your notebook here.

This matrix clearly shows:

- Strong diagonal dominance
- Minimal off-diagonal confusion
- Particularly sharp separation for stained_glass, gargoyle, and dome classes

The hardest classes (apse, flying buttress) tend to exhibit high intra-class variation and share textures or shapes with nearby architectural categories. Apse images often contain altars or domes, creating overlapping feature distributions. Flying buttresses vary significantly in orientation and scale, making them challenging for both scratch and pretrained models.

The pretrained ResNet18 resolves many of these confusions because ImageNet features capture general geometric primitives (edges, curves, repeating textures) that transfer well to architectural domains.

8. Conclusion

This study compared three neural network approaches for 10-class architectural scene classification. Our custom TinyDSCNN achieved strong results despite its small size, demonstrating the effectiveness of depthwise-separable convolutions and SE blocks on low-resolution images.

However, ResNet18 pretrained with fine-tuning achieved the best overall performance with 93.15 percent test accuracy and 98 percent top-2 accuracy, significantly improving minority-class performance. The confusion matrix confirms robust and consistent predictions across nearly all classes.

Transfer learning provides a decisive advantage for this dataset, while the TinyDSCNN remains a highly efficient alternative suitable for limited-compute scenarios.

9. Limitations and future improvements

One limitation of this study is the use of low-resolution 64×64 images, which can make it difficult for the models especially deeper networks to capture finer architectural details. Using higher resolutions such as 128×128 , or incorporating multi-scale inputs, would likely improve performance on visually subtle classes.

Another potential improvement lies in the data augmentation strategy. While our current augmentations help generalisation, more advanced techniques like RandAugment, CutMix, or Mixup could further increase robustness, particularly for under-represented classes.